

DATA STRUCTURES PRACTICE QUESTIONS

70 Questions – Complexity, Arrays, Searching & Sorting, Strings,
Linked Lists (Singly/Doubly/Circular), Stacks & Applications

Attempt without notes. Write code and dry-run. Analyse time & space complexity for every coding answer.

1–2. Complexity Analysis

- Q1. Define Big-O notation. Why do we ignore constants and lower-order terms?
- Q2. Arrange in increasing order of complexity: $O(n!)$, $O(n \log n)$, $O(2^n)$, $O(n^2)$, $O(\log n)$, $O(n)$.
- Q3. What is the time complexity of the nested loop: for $i \leftarrow 1$ to n ; for $j \leftarrow 1$ to i ; ... ?
- Q4. Differentiate between space complexity and auxiliary space with an example.
- Q5. What is the auxiliary space of recursive factorial vs iterative factorial?
- Q6. Give the time and space complexity of Merge Sort and Quick Sort (average & worst).

3–5. Arrays & Operations

- Q7. Why is random access $O(1)$ in an array but $O(n)$ in a linked list?
- Q8. Write code to reverse an array in-place. State its time and space complexity.
- Q9. How do you insert an element at index k in a fixed-size array (with logical size)? Complexity?
- Q10. Write a function to delete the first occurrence of a key from an array (shift elements).
- Q11. Explain prefix-sum technique. How does it help range-sum queries?

4. 2D / Multidimensional Arrays

- Q12. How is a 2D array stored in memory? What do `mat.length` and `mat[i].length` represent?
- Q13. Write code to find the sum of both diagonals of a square matrix.
- Q14. What is a jagged array? Give a short Java example.

6. Linear & Binary Search

- Q15. Write iterative Binary Search. What is its time complexity and precondition?
- Q16. Write recursive Binary Search. What is the auxiliary space?
- Q17. When would you prefer Linear Search over Binary Search?
- Q18. How many comparisons does Binary Search need in the worst case for $n = 2^k$?

7. Sorting & Array Problems

- Q19. Write Bubble Sort and state its best / average / worst time complexity.
- Q20. Explain Selection Sort in 3–4 lines. Is it stable?
- Q21. Using sorting + two pointers, check whether any two elements sum to a target. Complexity?
- Q22. What sorting algorithm does `Arrays.sort` use for `int[]` vs `Object[]` in Java?

8–10. Strings

- Q23. Why is String immutable? How does this enable the String Pool?
- Q24. Write a naïve pattern-search function. What is its worst-case complexity?
- Q25. Check whether two strings are anagrams. Give an efficient approach.
- Q26. Find the first non-repeating character in a string. Complexity?
- Q27. Reverse words in a sentence (e.g. "hello world" $\rightarrow$ "world hello").
- Q28. Why prefer `StringBuilder` over repeated String concatenation inside a loop?

11. Singly Linked List

- Q29. Draw / describe a singly linked list node. Write the Node class.
- Q30. Implement insert at head and insert at end. Compare their complexities.
- Q31. How do you find the length of a singly linked list?
- Q32. Delete a node given only a reference to that node (not the head). Constraint?

12. Doubly Linked List

- Q33. What extra field does a doubly-linked node have? Advantage over singly linked?

Q34. Write insert at head for a doubly linked list (update both next and prev).

Q35. How can you traverse a doubly linked list backward?

13. Circular Linked List

Q36. How does a circular singly linked list differ from a normal one?

Q37. Write a loop that prints each node of a circular list exactly once.

Q38. Name one classic problem that uses a circular list (hint: Josephus).

14–15. Linked List Operations & Classic Problems

Q39. Find the middle of a linked list in one pass (tortoise-hare). Write code.

Q40. Find the nth node from the end in one pass. Explain the two-pointer technique.

Q41. Reverse a singly linked list iteratively. Draw the pointer changes for 1→2→3.

Q42. Detect a cycle using Floyd's algorithm. Prove why meeting implies a cycle.

Q43. Merge two sorted linked lists into one sorted list. Complexity?

Q44. Remove duplicates from a sorted linked list.

Q45. Predict the output / effect:

```
Node a = new Node(1), b = new Node(2), c = new Node(3);  
a.next = b; b.next = c; c.next = b; // what does hasCycle return?
```

16–17. Stack ADT & Implementation

Q46. Define Stack ADT. List the four fundamental operations and their ideal complexity.

Q47. Implement a stack using an array (push, pop, peek, isEmpty). Handle overflow/underflow.

Q48. Implement a stack using a linked list. Why is there no overflow (until memory exhaustion)?

Q49. Compare array-based vs linked-list-based stack (time, space, cache locality).

18. Stack Applications

Q50. Write a function to check whether a string of brackets is balanced. Complexity?

Q51. Explain how a stack is used to evaluate a postfix expression. Give a dry-run for "2 3 * 4 +".

Q52. What is the function call stack? What happens on a recursive call and on return?

Q53. Convert the infix expression (A+B)*C to postfix using a stack (show steps).

Q54. Name three real-world applications of stacks besides expression evaluation.

Bonus / Mixed

Q55. Given an array, find the maximum subarray sum (Kadane's algorithm). Complexity?

Q56. Rotate an array to the right by k positions in-place (or using extra space).

Q57. Check if a linked list is a palindrome (use a stack or reverse second half).

Q58. Implement two stacks in a single array. Explain push/pop for both.

Q59. What is the time complexity of inserting at the beginning of: Array vs Singly LL vs Doubly LL?

Q60. Dry-run Floyd's cycle detection on a list: 1→2→3→4→2 (cycle back to 2).

Q61. Sort an array of 0s, 1s and 2s in one pass (Dutch National Flag).

Q62. Find the intersection point of two singly linked lists (if any).

Q63. Why is Quick Sort often faster than Merge Sort in practice despite same average complexity?

Q64. Implement a queue using two stacks. Analyse amortized complexity of enqueue/dequeue.

Q65. Given a string, remove all adjacent duplicates repeatedly (use a stack). Example: "abbaca" → "ca".

Q66. Write Binary Search that returns the leftmost insertion point (lower_bound style).

Q67. Detect and remove a cycle in a linked list (find start of cycle then break it).

Q68. Next Greater Element for each array element using a stack. Complexity?

Q69. Explain the difference between stable and unstable sorting with an example.

Q70. Design a stack that supports push, pop, top and getMin in O(1) time.

After solving, verify against the DSA Concept Guide. Always state Time & Space complexity for coding answers. Practice drawing linked-list pointer changes and stack states on paper.

All the best for your placements!